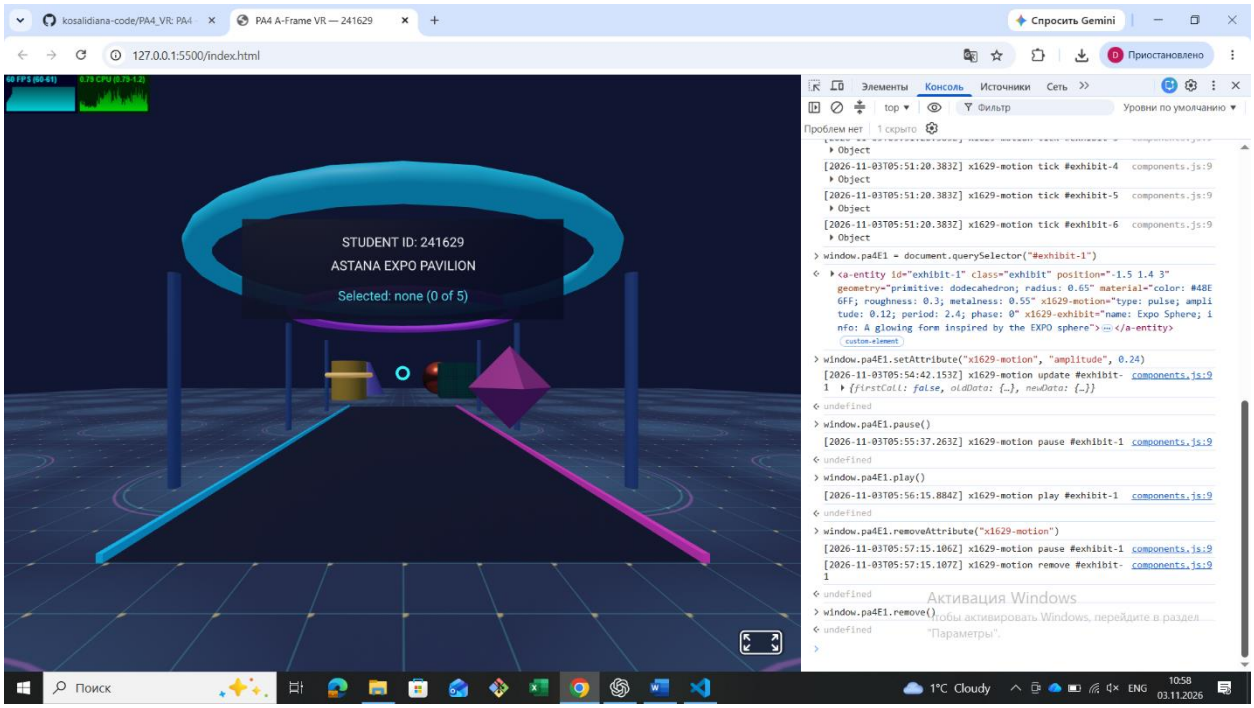
PA4 — A-Frame VR Scene with Custom Component

Prepared by Kossaly Diana

Student ID: 241629

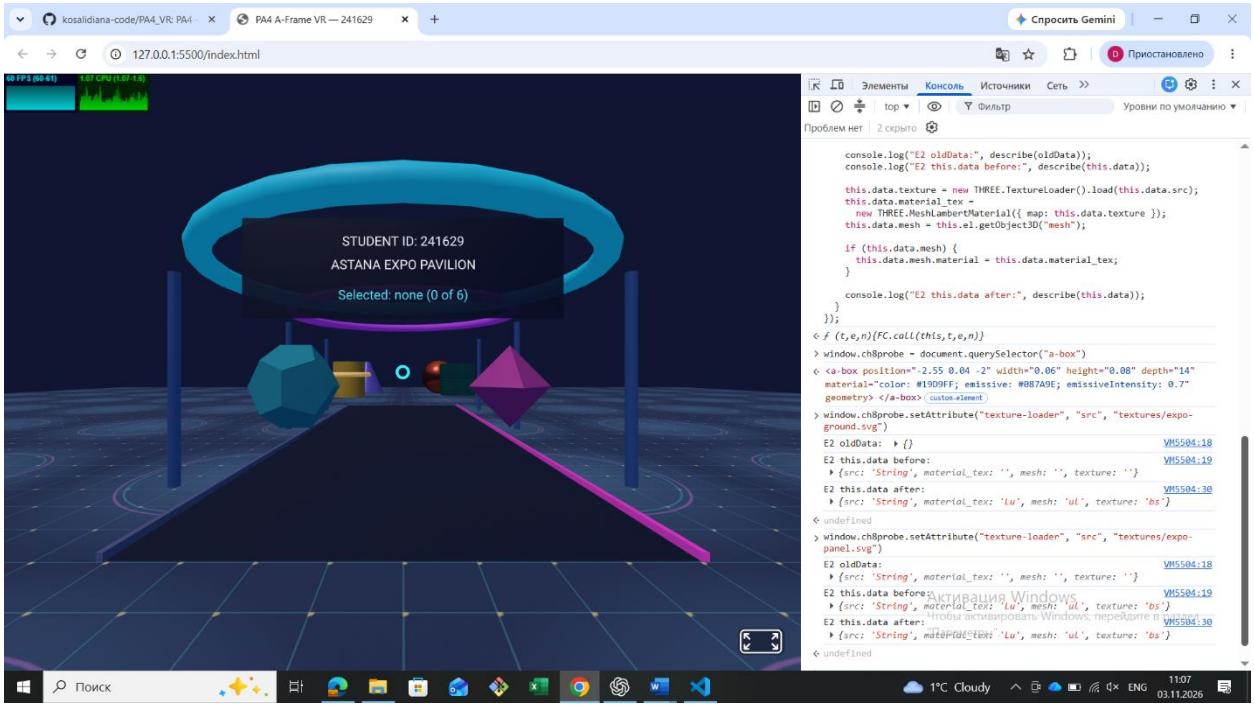


E1 — Component life cycle

Prediction: The motion component should run its life-cycle methods when an exhibit is created, updated, paused, resumed, or removed.

Result: Timestamped console logs showed `init`, the first `update`, `play`, and `tick` after page load. Changing the amplitude triggered another `update`. Calling `pause()` and `play()` produced the corresponding lifecycle logs. Removing the motion attribute triggered `pause` and `remove`. Removing the exhibit then changed the gallery count from six to five.

Explanation: On the first `update`, `oldData` is empty because the component has no previous data. When I changed the amplitude from 0.12 to 0.24, `oldData` contained the previous component values and the new data contained the updated amplitude. This lets the component compare old and new settings.

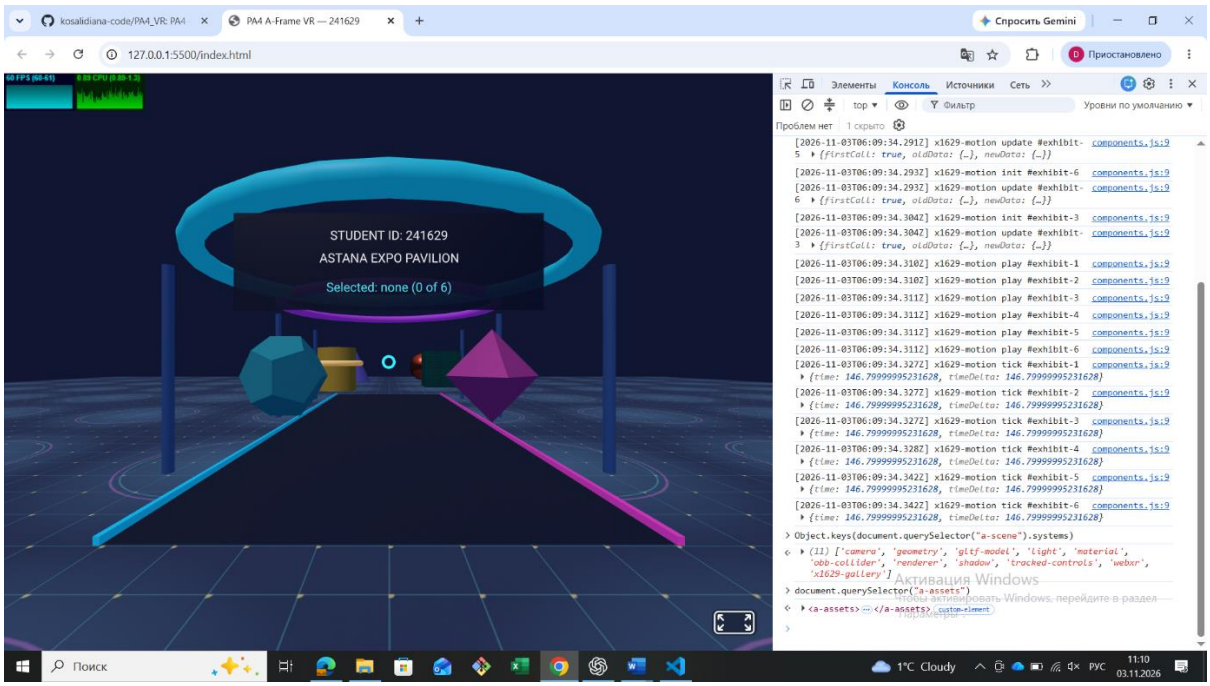


E2a — The Chapter 8 texture-loader

Prediction: Changing the texture source should trigger the component's `update` method and change its data.

Result: The console test showed that the Chapter 8 component placed the texture, material, and mesh in `this.data`. After changing `src`, the next update began with those fields at their schema defaults and then recreated the runtime objects.

Explanation: The component's schema data is managed from its HTML attributes, so runtime objects should not be stored there. The schema should hold configuration such as the source and repeat values; runtime objects should be kept on the component instance, such as `this.texture`, `this.material`, and `this.mesh`.

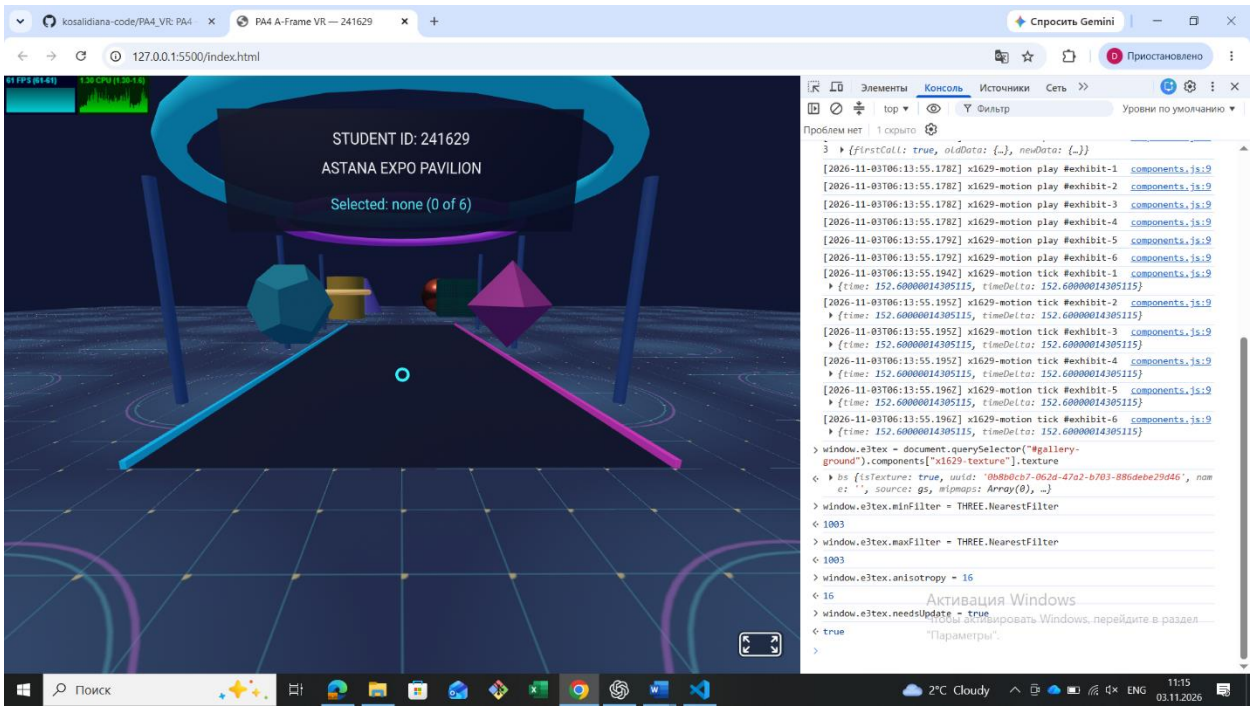


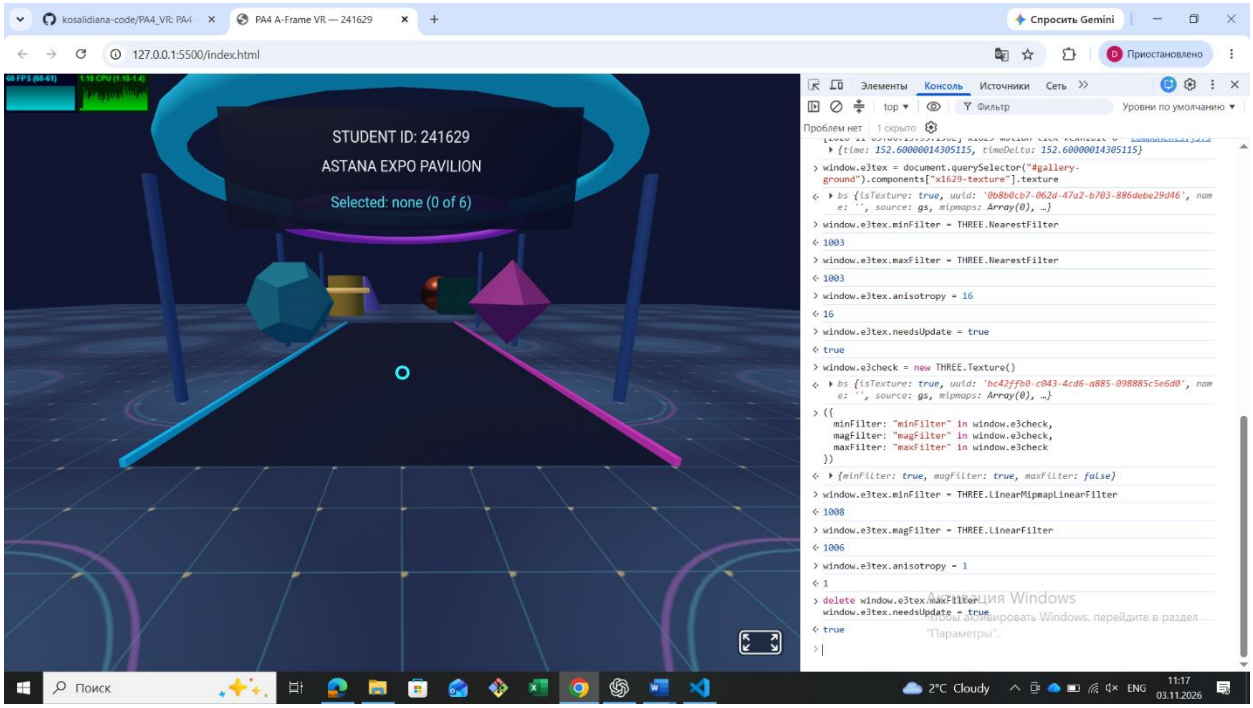
E2b — A-Frame systems

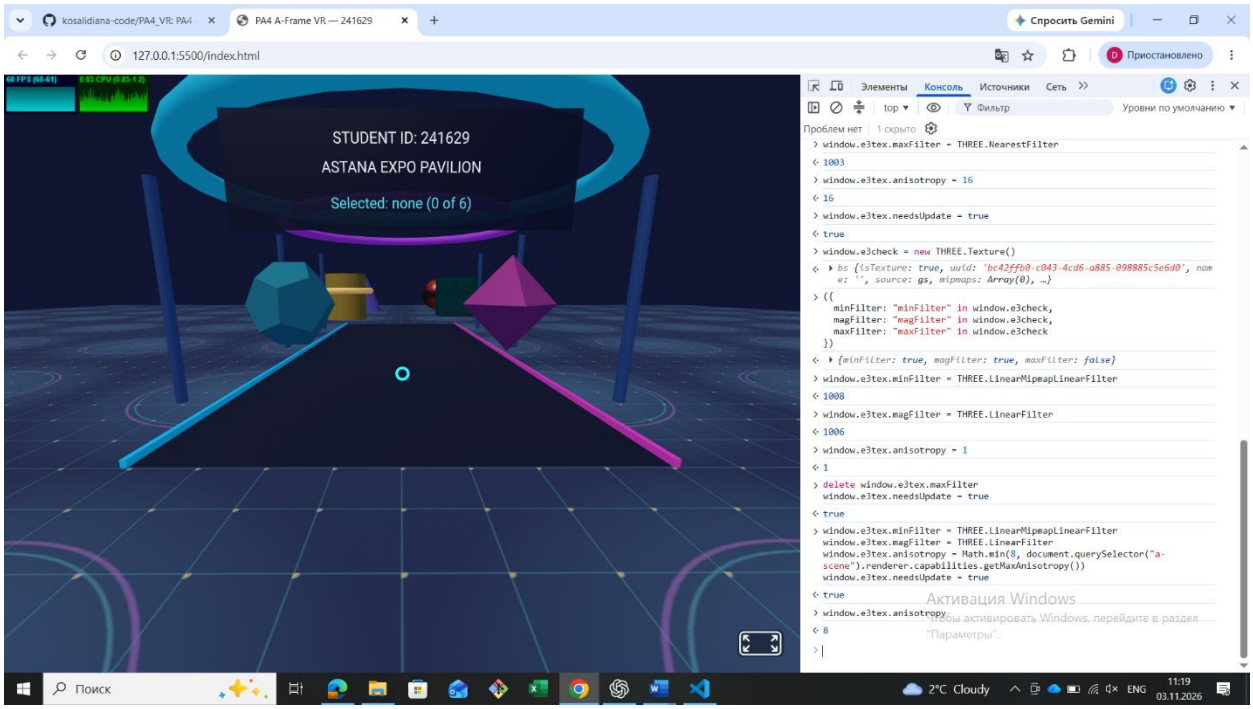
Prediction: The asset manager and a system registered with `AFRAME.registerSystem` should provide different services to the scene.

Result: The console listed `x1629-gallery` in `scene.systems`. Querying `a-assets` returned an `<a-assets>` element.

Explanation: `<a-assets>` provides A-Frame's built-in asset preloading service. `AFRAME.registerSystem` creates a scene-level system instance for shared application logic. My `x1629-gallery` system tracks exhibits, selection, and keyboard controls. In this sense the chapter calls `<a-assets>` a system by its role, though it is not registered through the custom-system API.

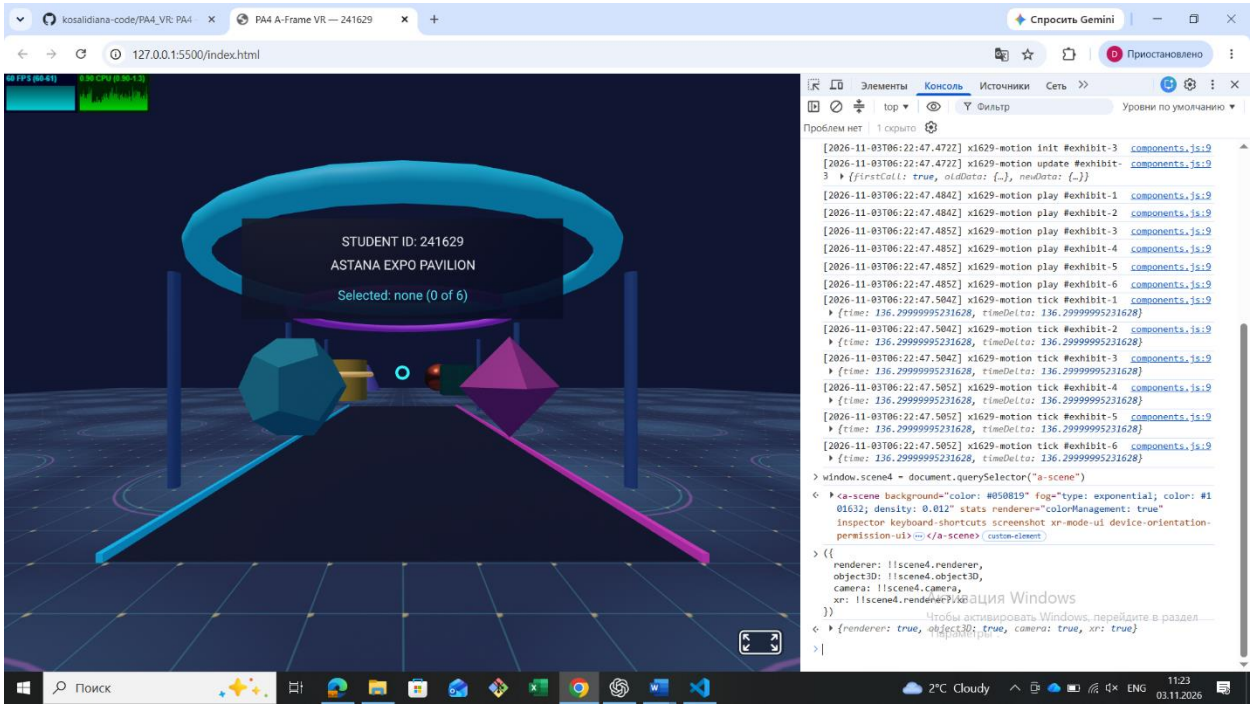


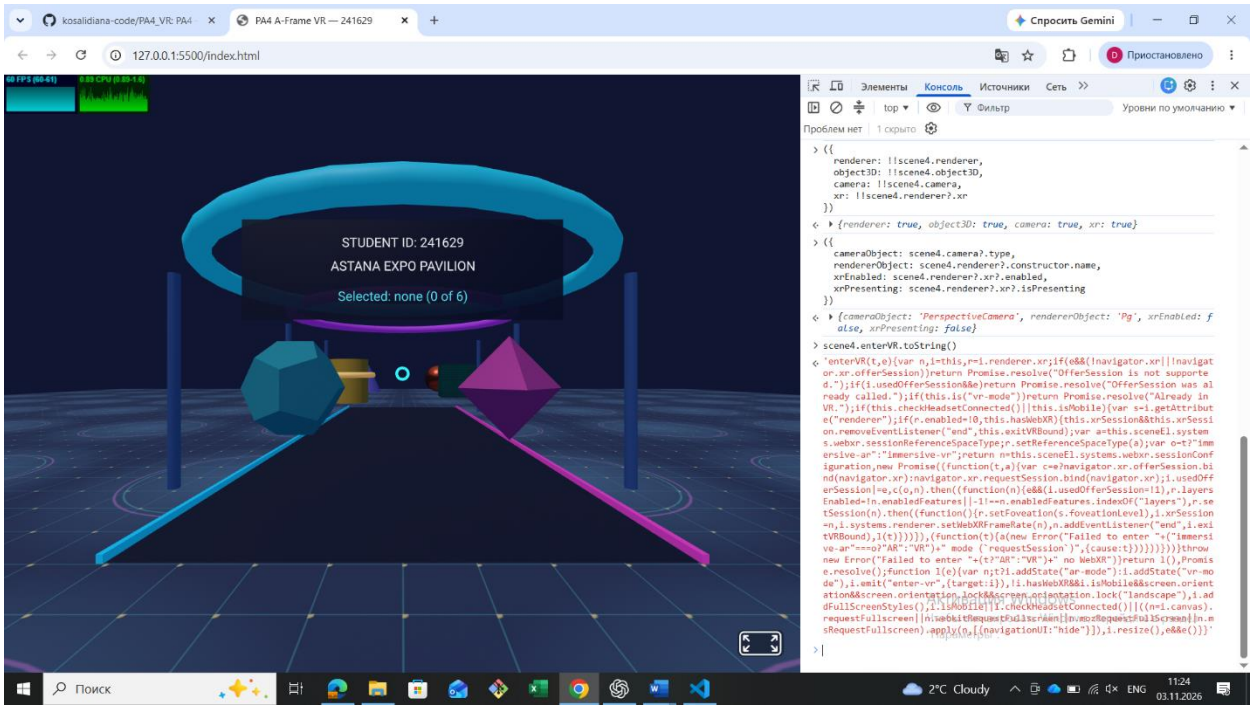




E3 — Texture filtering

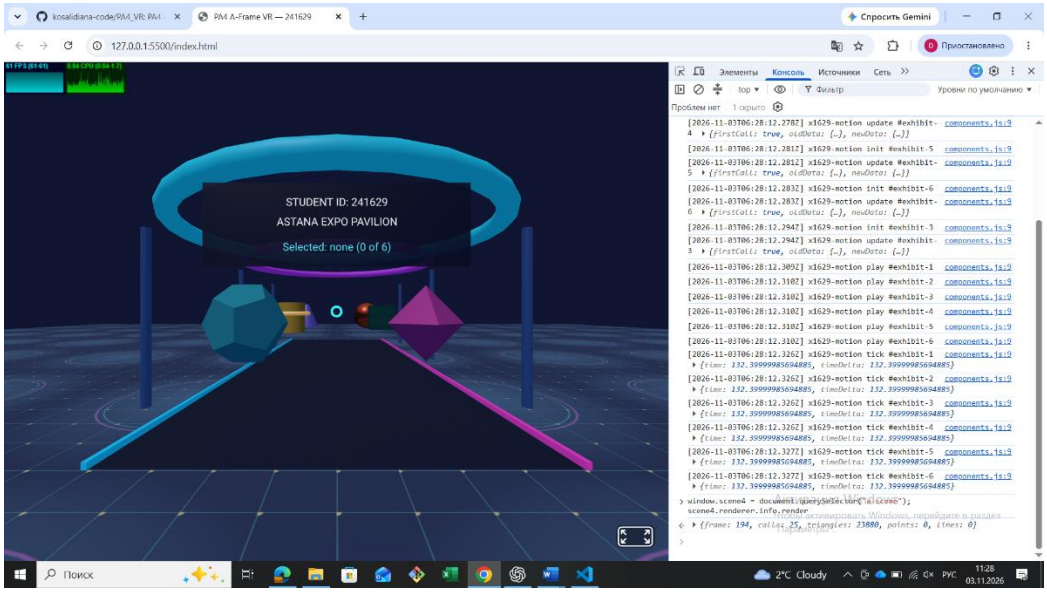
I compared the Chapter 8 filter settings, Three.js defaults, and my selected settings while viewing the ground texture at a low angle. The Chapter 8 setup used NearestFilter and anisotropy 16; the texture looked more pixelated. Three.js defaults use LinearMipmapLinearFilter for minification and LinearFilter for magnification, with anisotropy 1. My final setup uses those default filters and anisotropy 8, which made the angled ground pattern clearer in my comparison. Minification filtering applies when a texture appears smaller on screen; magnification filtering applies when it appears larger. Mipmaps provide smaller versions of a texture to reduce aliasing when viewed from farther away. Anisotropic filtering improves texture detail at oblique angles. The console check also showed that `maxFilter` is not a standard texture property.

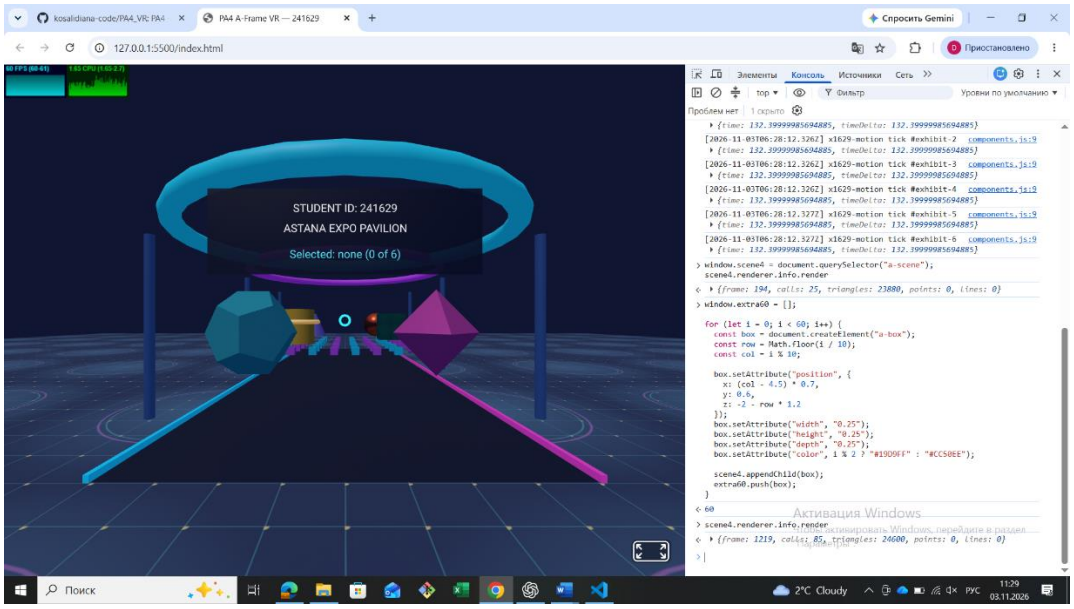


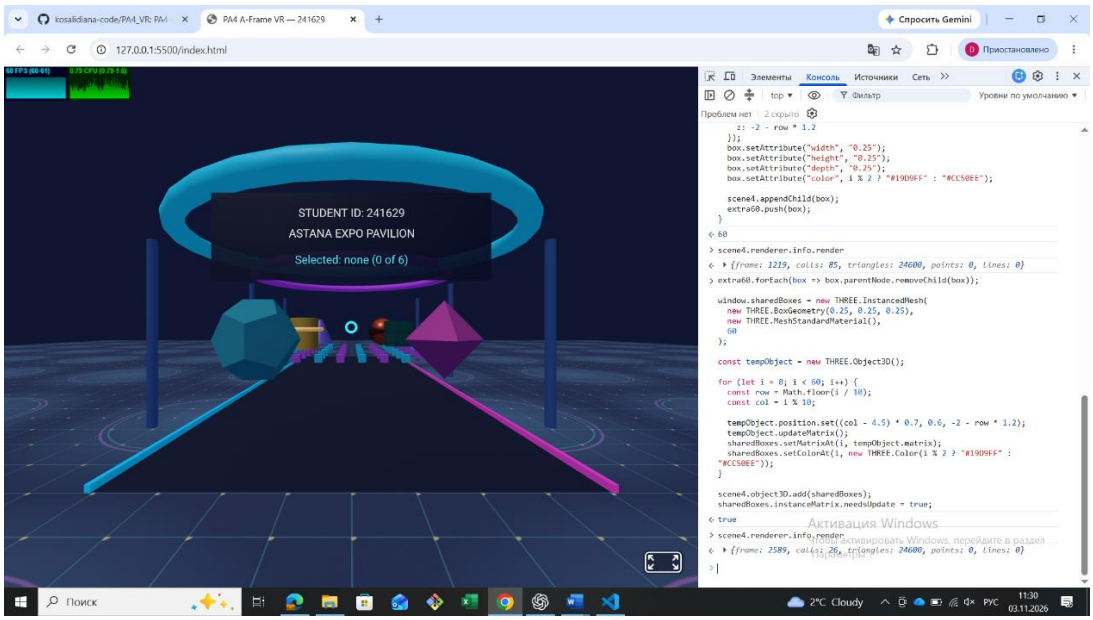


E4 — A-Frame scene and WebXR

The console inspection confirmed that the A-Frame scene has a Three.js `WebGLRenderer`, an `Object3D` scene graph, a `PerspectiveCamera`, and an XR interface. Before entering VR, `renderer.xr.enabled` and `renderer.xr.isPresenting` were both false. Inspecting `scene.enterVR()` showed that A-Frame checks for `navigator.xr`, requests a session with `navigator.xr.requestSession(...)`, and connects the session to its WebXR rendering system. In my PA3 project, I worked more directly with WebXR and the renderer; A-Frame provides a higher-level scene and entity-component structure around that process. I would choose raw Three.js when I need direct control over rendering and scene setup. I would choose A-Frame for a scene that benefits from declarative HTML entities and reusable components.

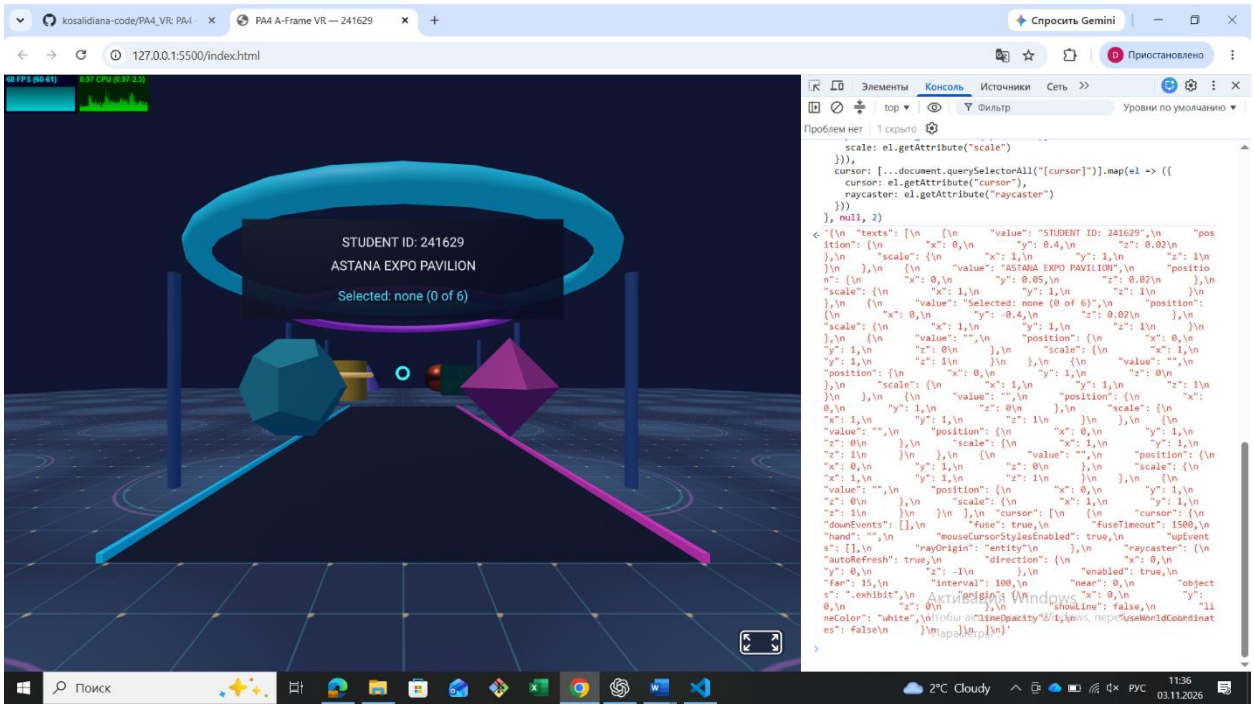

STUDENT ID: 241629
ASTANA EXPO PAVILION
Selected: none (0 of 6)


STUDENT ID: 241629
ASTANA EXPO PAVILION
Selected: none (0 of 6)


STUDENT ID: 241629
ASTANA EXPO PAVILION
Selected: none (0 of 6)

E5 — Draw-call performance

The baseline scene rendered at about 61 FPS with 25 draw calls and 23,880 triangles. I added 60 separate box exhibits; the result was about 60 FPS, 85 draw calls, and 24,600 triangles. I then replaced those boxes with one Three.js `InstancedMesh`. The result was about 60 FPS, 26 draw calls, and 24,600 triangles. Instancing reduced the draw-call count by 59 compared with the baseline containing the 60 extra objects, while preserving the same triangle count for those boxes. This matters in VR because the renderer draws the scene for two eyes, so reducing repeated draw submissions can lower CPU overhead. These figures are from a temporary browser-console experiment.



E6 — Comfort rationale

The pavilion uses a 10 m corridor with two exhibit rows separated by 3 m, leaving a clear central viewing path. This spacing gives visitors room to look toward exhibits without needing to move close to them. The title and status text use scale 1 and are presented on a camera-facing panel, giving them a large, readable angular size in the current view; the high-contrast text helps distinguish it from the dark scene. Gaze selection uses a 1.5-second fuse with a visible reticle and selection counter. A possible comfort risk is accidental selection while a visitor looks at an exhibit for a long time. The dwell delay and visible selection feedback help reduce unintended activation. The scene avoids forced camera movement, so visitors control their own viewing direction.